

FastAPI Tutorial

A Complete Guide

39 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead:

<https://snehaliteng.github.io/tutorials/fastapi-tutorial/>

[← Back to Tutorials](#)

1. Home

Course Overview

This course takes you from zero to production-ready with FastAPI across 39 chapters. You'll learn to build REST APIs, handle parameters, validate data, connect to databases, add real-time features, and deploy to the cloud.

Prerequisites

- Python 3.8+ installed
- Basic Python knowledge (functions, classes, types)
- A code editor (VS Code recommended)

Setup

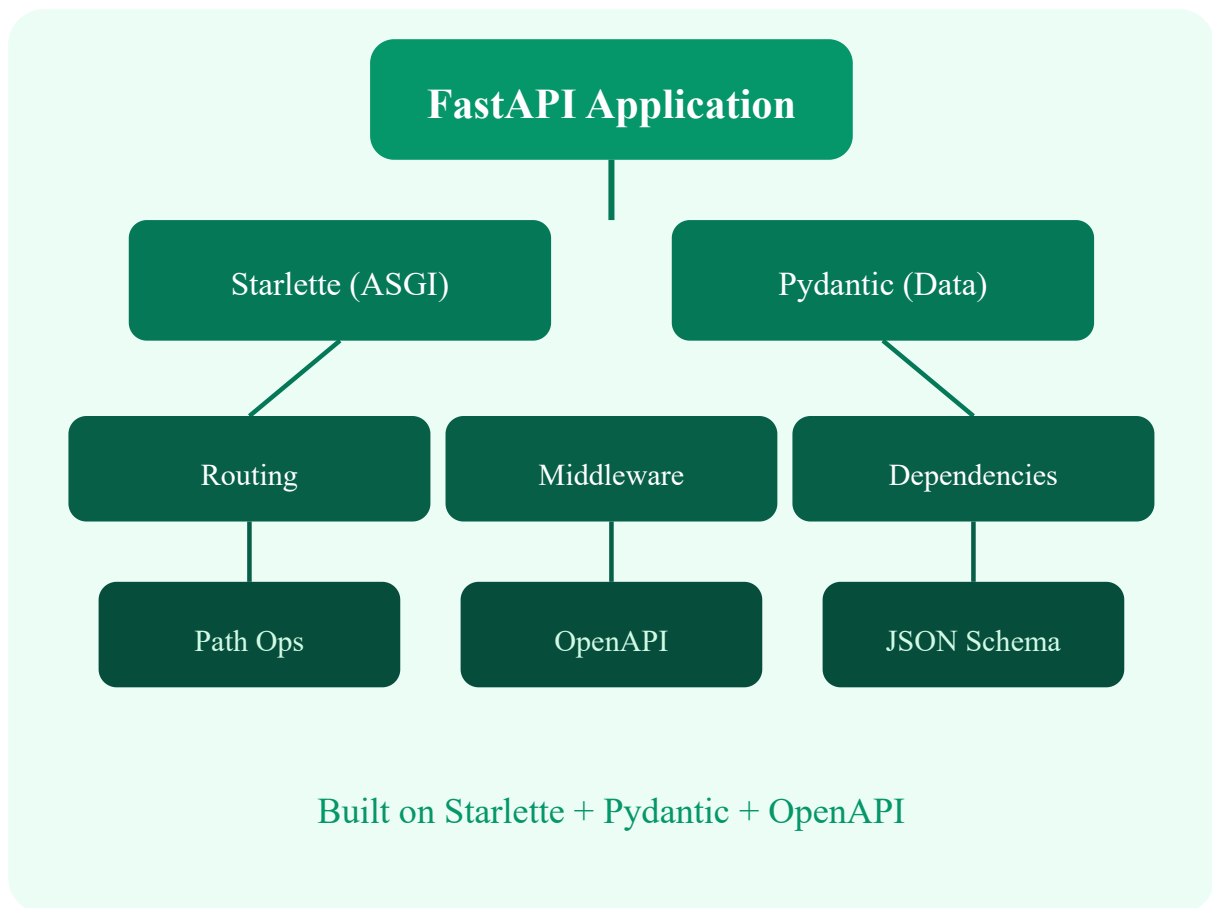
```
# Create project directory
mkdir fastapi-course && cd fastapi-course

# Create virtual environment
python -m venv venv

# Windows
venv\Scripts\activate

# macOS/Linux
source venv/bin/activate

# Install FastAPI
pip install fastapi uvicorn
```



 **Exercise:** Create a virtual environment, install FastAPI and uvicorn, and verify with `pip list | findstr fastapi`.

2. Introduction

What is FastAPI?

FastAPI is a modern, high-performance Python web framework for building APIs. It's built on Starlette (for async) and Pydantic (for data validation), and automatically generates OpenAPI documentation.

Why FastAPI?

Feature	FastAPI	Flask	Django
Async support	Native	Extension	3.1+
Auto docs	Yes (OpenAPI)	Third-party	DRF
Data validation	Pydantic built-in	Manual	DRF serializers
Performance	Node.js/Go-level	WSGI	WSGI
Type hints	Required	Optional	Optional

Key Features

- High performance — on par with Node.js and Go
- Fast to code — 200-300% faster development
- Fewer bugs — ~40% fewer human-induced errors
- Automatic interactive API docs (Swagger + ReDoc)
- Standards-based — OpenAPI + JSON Schema

 **Exercise:** Research a FastAPI success story (e.g., Microsoft, Uber, Netflix). Write a paragraph on why they chose FastAPI.

[← Back to Tutorials](#)

3. Hello World

First FastAPI App

```
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello World"}

@app.get("/items/{item_id}")
def read_item(item_id: int, q: str = None):
    return {"item_id": item_id, "q": q}
```

Run the Server

```
# Terminal
uvicorn main:app --reload

# Output
INFO:      Uvicorn running on http://127.0.0.1:8000
INFO:      Application startup complete.
```

Test the API

Open `http://127.0.0.1:8000` in your browser. You should see `{"message": "Hello World"}`. Try `http://127.0.0.1:8000/items/42?q=test`.

 **Exercise:** Create the Hello World app, run it with uvicorn, and verify the root and items endpoints respond correctly. Add a third endpoint returning your name.

4. OpenAPI & Swagger

Automatic Documentation

FastAPI automatically generates an OpenAPI schema from your code. Visit:

- **Swagger UI:** <http://127.0.0.1:8000/docs>
- **ReDoc:** <http://127.0.0.1:8000/redoc>
- **OpenAPI JSON:** <http://127.0.0.1:8000/openapi.json>

Customizing Metadata

```
app = FastAPI(
    title="My API",
    description="A sample FastAPI application",
    version="1.0.0",
    contact={"name": "Snehal", "email": "snehal@example.com"},
    license_info={"name": "MIT"},
)

@app.get("/items/{item_id}",
        summary="Get an item",
        description="Retrieve an item by its ID",
        tags=["items"],
        deprecated=False)
def read_item(item_id: int): ...
```

OpenAPI Schema

```
# View the raw schema at /openapi.json
{
  "openapi": "3.1.0",
  "info": {"title": "My API", "version": "1.0.0"},
  "paths": {
    "/items/{item_id}": {
      "get": {
        "parameters": [...],
        "responses": {...}
      }
    }
  }
}
```

 **Exercise:** Run your app and open /docs. Add title, description, and version metadata. Verify the Swagger UI reflects your changes.

5. Uvicorn Basics

What is Uvicorn?

Uvicorn is an ASGI (Asynchronous Server Gateway Interface) server for Python. It's the recommended server for running FastAPI applications.

Common Options

```
# Basic usage
uvicorn main:app --reload

# Custom host and port
uvicorn main:app --host 0.0.0.0 --port 8080

# Number of workers (production)
uvicorn main:app --workers 4

# Log level
uvicorn main:app --log-level debug

# SSL
uvicorn main:app --ssl-keyfile key.pem --ssl-certfile cert.pem
```

Programmatic Startup

```
# run.py
import uvicorn

if __name__ == "__main__":
    uvicorn.run(
        "main:app",
        host="0.0.0.0",
        port=8000,
        reload=True,
        log_level="info"
    )
```

Lifecycle Events

Uvicorn fires startup and shutdown events:

```
@app.on_event("startup")
async def startup():
    print("Server is starting...")

@app.on_event("shutdown")
async def shutdown():
    print("Server is shutting down...")
```

 **Exercise:** Run your app with `--reload`, change a line in `main.py`, and observe the auto-reload. Then run on port 8080 with 2 workers.

[← Back to Tutorials](#)

6. Type Hints

Type hints are at the core of FastAPI. They provide data validation, serialization, and automatic documentation.

Basic Types

```
# Python type hints
name: str = "FastAPI"
count: int = 42
price: float = 19.99
is_active: bool = True

# Function signatures
def greet(name: str) -> str:
    return f"Hello {name}"
```

Optional & Union

```
from typing import Optional, Union, List, Dict

# Optional (can be None)
def get_item(item_id: int, q: Optional[str] = None):
    ...

# Union (multiple possible types)
def process(value: Union[int, str]) -> str:
    return str(value)
```

Collections

```
from typing import List, Dict, Set, Tuple

items: List[int] = [1, 2, 3]
metadata: Dict[str, str] = {"key": "value"}
unique: Set[int] = {1, 2, 3}
pair: Tuple[int, str] = (1, "one")

# FastAPI uses these for query params
@app.get("/items/")
def list_items(ids: List[int] = None):
    return {"ids": ids}
```

 **Exercise:** Write a function with type hints that takes a list of integers and returns the sum. Add an optional multiplier parameter.

[← Back to Tutorials](#)

7. IDE Support

VS Code Setup

```
# Install extensions:
# - Python (Microsoft)
# - Pylance
# - FastAPI Snippets (by Manas)

# Settings.json
{
  "python.linting.enabled": true,
  "python.linting.pylintEnabled": false,
  "python.linting.mypyEnabled": true,
  "python.formatting.provider": "black",
  "editor.formatOnSave": true
}
```

Pylance Features

- Type checking with type hints
- Auto-completion for FastAPI methods
- Parameter hints from Pydantic models
- Inline errors for type mismatches
- Go to Definition for dependencies

PyCharm Setup

PyCharm Professional has built-in FastAPI support including endpoint navigation, run configurations, and test generation.

Benefits

With type hints, your IDE can catch errors before you run the code — incorrect types, missing parameters, wrong return types. This is a major productivity boost.

 **Exercise:** Configure VS Code or PyCharm for FastAPI. Write a route with a typo and verify your IDE catches it before running.

[← Back to Tutorials](#)

8. REST Architecture

HTTP Methods

Method	CRUD	FastAPI Decorator
GET	Read	@app.get()
POST	Create	@app.post()
PUT	Update (full)	@app.put()
PATCH	Update (partial)	@app.patch()
DELETE	Delete	@app.delete()

RESTful Design Principles

```
# Resources as nouns, not verbs
# GOOD
GET    /users          # List users
GET    /users/{id}     # Get user
POST   /users          # Create user
PUT    /users/{id}    # Update user
DELETE /users/{id}     # Delete user

# BAD
GET    /get_users
POST   /create_user
POST   /delete_user
```

Status Codes

```
from fastapi import FastAPI, status

@app.post("/items/", status_code=status.HTTP_201_CREATED)
def create_item():
    return {"item": "created"}

@app.delete("/items/{id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_item(id: int):
    return # No body for 204
```

 **Exercise:** Design RESTful endpoints for a blog API (posts, comments, users). Implement the GET and POST endpoints using FastAPI decorators.

9. Path Parameters

```
from fastapi import FastAPI

app = FastAPI()

# Path parameter with type
@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}

# Multiple path params
@app.get("/users/{user_id}/orders/{order_id}")
def get_order(user_id: int, order_id: int):
    return {"user_id": user_id, "order_id": order_id}

# Path parameter order matters – static before dynamic
@app.get("/users/me")          # static
def get_current_user(): ...

@app.get("/users/{user_id}")   # dynamic
def get_user(user_id: int): ...

# Enum path parameter
from enum import Enum

class ModelName(str, Enum):
    alexnet = "alexnet"
    resnet = "resnet"
    lenet = "lenet"

@app.get("/models/{model_name}")
def get_model(model_name: ModelName):
    return {"model": model_name}
```



Exercise: Create endpoints with path parameters for a library API:

`/books/{book_id}` , `/authors/{author_id}/books/{book_id}` . Test them in Swagger UI.

10. Query Parameters

```
from fastapi import FastAPI

app = FastAPI()

# Optional query param with default
@app.get("/items/")
def list_items(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}

# Optional query param (can be None)
@app.get("/items/{item_id}")
def read_item(item_id: int, q: str = None, short: bool = False):
    return {"item_id": item_id, "q": q, "short": short}

# Required query param (no default)
@app.get("/search/")
def search(q: str):
    return {"query": q}

# Type conversion
@app.get("/users/")
def get_users(active: bool = True, page: int = 1):
    return {"active": active, "page": page}

# Multiple values
from typing import List

@app.get("/filter/")
def filter_items(tags: List[str] = None):
    return {"tags": tags}

# URL: /filter/?tags=a&tags=b&tags=c
```

 **Exercise:** Create an endpoint that accepts `category` (optional), `min_price` (optional float), and `sort_by` (default "name") query parameters.

[← Back to Tutorials](#)

11. Parameter Validation

```
from fastapi import FastAPI, Query, Path

app = FastAPI()

# Query parameter validation
@app.get("/items/")
def read_items(
    q: str = Query(None, min_length=3, max_length=50),
    price: float = Query(..., gt=0, le=1000)
):
    return {"q": q, "price": price}

# Path parameter validation
@app.get("/items/{item_id}")
def read_item(
    item_id: int = Path(..., ge=1, title="Item ID")
):
    return {"item_id": item_id}

# String patterns (regex)
@app.get("/users/")
def search_users(
    code: str = Query(..., regex="^[A-Z]{3}\\d{3}$")
):
    return {"code": code}

# Multiple constraints
@app.get("/products/")
def list_products(
    page: int = Query(1, ge=1),
    size: int = Query(10, ge=1, le=100),
    sort: str = Query("name", regex="^(name|price|date)$")
):
    return {"page": page, "size": size, "sort": sort}
```


 **Exercise:** Create an endpoint with a path param `user_id` (≥ 1) and a query param `email` (valid email regex pattern). Test validation errors in Swagger.

[← Back to Tutorials](#)

12. Pydantic Models

```
from pydantic import BaseModel, Field, EmailStr
from typing import Optional

class Item(BaseModel):
    name: str
    price: float = Field(..., gt=0, description="Price must be positive")
    is_offer: bool = False
    tax: Optional[float] = None

class User(BaseModel):
    username: str = Field(..., min_length=3, max_length=50)
    email: EmailStr
    full_name: Optional[str] = None

# Validator
from pydantic import validator

class Order(BaseModel):
    items: List[Item]
    discount: float = 0.0

    @validator("discount")
    def check_discount(cls, v):
        if v < 0 or v > 100:
            raise ValueError("Discount must be 0-100")
        return v
```

 **Exercise:** Create a Pydantic model for a `Product` with name, price (>0), category, and an optional description. Add a validator for price.

[← Back to Tutorials](#)

13. Request Body

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str = None
    price: float
    tax: float = None

# Single body parameter
@app.post("/items/")
def create_item(item: Item):
    return {"name": item.name, "price": item.price}

# Body + path + query
@app.put("/items/{item_id}")
def update_item(
    item_id: int,
    item: Item,
    q: str = None
):
    return {"item_id": item_id, **item.dict(), "q": q}
```

Multiple Body Parameters

```
class User(BaseModel):
    username: str
    full_name: str = None

@app.put("/items/{item_id}")
def update_item(
    item_id: int,
    item: Item,
    user: User,
    importance: int = Body(..., ge=1)
):
    return {"item_id": item_id, "item": item, "user": user}
```

 **Exercise:** Create a POST endpoint that accepts a `User` model (username, email, age) as the request body. Return the user data with a welcome message.

[← Back to Tutorials](#)

14. HTML Templates

```
pip install jinja2
```

```
from fastapi import FastAPI, Request
from fastapi.templating import Jinja2Templates
from fastapi.responses import HTMLResponse

app = FastAPI()
templates = Jinja2Templates(directory="templates")

@app.get("/", response_class=HTMLResponse)
def home(request: Request):
    return templates.TemplateResponse(
        "index.html",
        {"request": request, "title": "FastAPI App"}
    )

@app.get("/items/{id}", response_class=HTMLResponse)
def item_detail(request: Request, id: int):
    return templates.TemplateResponse(
        "item.html",
        {"request": request, "item_id": id}
    )
```

Template File (templates/index.html)

```
<!DOCTYPE html>
<html>
<head><title>{{ title }}</title></head>
<body>
  <h1>Welcome to {{ title }}</h1>
  <p>This page is rendered with Jinja2.</p>
</body>
</html>
```

 **Exercise:** Create a "templates" folder with an index.html. Render it from FastAPI passing a list of items. Display them in a loop using Jinja2 syntax.

[← Back to Tutorials](#)

15. Static Files

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles

app = FastAPI()

# Mount static files directory
app.mount("/static", StaticFiles(directory="static"), name="static")
```

Folder Structure

```
project/
├─ main.py
├─ static/
│   ├─ css/
│   │   └─ style.css
│   ├─ js/
│   │   └─ app.js
│   └─ images/
│       └─ logo.png
└─ templates/
    └─ index.html
```

Using in Templates

```
<!-- templates/index.html -->
<link rel="stylesheet" href="/static/css/style.css">
<script src="/static/js/app.js"></script>

```

 **Note:** Mounting static files happens after routes. If you mount before, FastAPI matches the mount path before checking routes. Order matters for path matching.

 **Exercise:** Create a static folder with a CSS file that styles your template. Mount it in FastAPI and verify the CSS is applied in the browser.

[← Back to Tutorials](#)

16. HTML Forms

```
from fastapi import FastAPI, Request, Form
from fastapi.templating import Jinja2Templates
from fastapi.responses import HTMLResponse, RedirectResponse

app = FastAPI()
templates = Jinja2Templates(directory="templates")

@app.get("/login", response_class=HTMLResponse)
def login_form(request: Request):
    return templates.TemplateResponse("login.html", {"request": request})

@app.post("/login")
def login(request: Request, username: str = Form(...), password: str = Form(...)):
    # Validate credentials
    if username == "admin" and password == "secret":
        return RedirectResponse(url="/dashboard", status_code=302)
    return templates.TemplateResponse(
        "login.html",
        {"request": request, "error": "Invalid credentials"}
    )
```

HTML Form Template

```
<form method="post" action="/login">
  <input type="text" name="username" placeholder="Username">
  <input type="password" name="password" placeholder="Password">
  <button type="submit">Login</button>
</form>
{% if error %}
  <p style="color:red">{{ error }}</p>
{% endif %}
```

 **Exercise:** Create a registration form with username, email, password, and confirm_password fields. POST to FastAPI, validate all fields, and return success/error.

[← Back to Tutorials](#)

17. Form Data

```
pip install python-multipart
```

```

from fastapi import FastAPI, Form

app = FastAPI()

@app.post("/submit/")
def submit_form(
    username: str = Form(...),
    password: str = Form(...),
    remember_me: bool = Form(False)
):
    return {
        "username": username,
        "remember_me": remember_me
    }

# Multiple form fields
@app.post("/register/")
def register(
    username: str = Form(..., min_length=3),
    email: str = Form(...),
    age: int = Form(..., ge=18),
    bio: str = Form(None, max_length=500)
):
    return {"username": username, "email": email}

# Form data with file
from fastapi import UploadFile, File

@app.post("/profile/")
def create_profile(
    name: str = Form(...),
    avatar: UploadFile = File(...)
):
    return {"name": name, "avatar": avatar.filename}

```

 **Note:** Form data uses `application/x-www-form-urlencoded`. Do NOT mix Form and JSON Body parameters in the same endpoint — use separate endpoints or Body parameters.

 **Exercise:** Create a form POST endpoint that accepts name, email, and age. Return a summary of the submitted data.

[← Back to Tutorials](#)

18. File Upload

```
pip install python-multipart
```

```

from fastapi import FastAPI, UploadFile, File
from fastapi.responses import JSONResponse

app = FastAPI()

# Single file upload
@app.post("/upload/")
async def upload_file(file: UploadFile = File(...)):
    contents = await file.read()
    return {
        "filename": file.filename,
        "content_type": file.content_type,
        "size": len(contents)
    }

# Save file to disk
import aiofiles

@app.post("/save/")
async def save_file(file: UploadFile = File(...)):
    async with aiofiles.open(f"uploads/{file.filename}", "wb") as f:
        await f.write(await file.read())
    return {"message": "File saved", "filename": file.filename}

# Multiple files
from typing import List

@app.post("/upload-multiple/")
async def upload_multiple(files: List[UploadFile] = File(...)):
    return {"filenames": [f.filename for f in files]}

# File validation
@app.post("/validated-upload/")
async def validated_upload(file: UploadFile = File(...)):
    if file.content_type not in ["image/jpeg", "image/png", "application/pdf"]:
        return JSONResponse(
            {"error": "Only JPEG, PNG, PDF allowed"},
            status_code=400
        )

```

```
)  
return {"filename": file.filename}
```

 **Exercise:** Create an endpoint that accepts a single image file, validates it's a JPEG/PNG, saves it to disk, and returns the file size and dimensions.

[← Back to Tutorials](#)

19. Cookie Parameters

```
from fastapi import FastAPI, Cookie
from fastapi.responses import JSONResponse

app = FastAPI()

# Read cookies
@app.get("/items/")
def read_items(ads_id: str = Cookie(None)):
    return {"ads_id": ads_id}

# Set cookies
@app.get("/set-cookie/")
def set_cookie():
    response = JSONResponse({"message": "Cookie set"})
    response.set_cookie(
        key="my_cookie",
        value="cookie_value",
        max_age=3600,
        httponly=True,
        samesite="lax"
    )
    return response

# Delete cookies
@app.get("/delete-cookie/")
def delete_cookie():
    response = JSONResponse({"message": "Cookie deleted"})
    response.delete_cookie("my_cookie")
    return response

# Multiple cookies
@app.get("/read-all-cookies/")
def read_all_cookies(
    session_id: str = Cookie(None),
    user_pref: str = Cookie(None)
```

```
):  
    return {"session_id": session_id, "user_pref": user_pref}
```

 **Exercise:** Create an endpoint that reads a "visited" cookie. If not present, set it with current timestamp. Return the visit count based on cookie history.

[← Back to Tutorials](#)

20. Header Parameters

```
from fastapi import FastAPI, Header
from typing import Optional

app = FastAPI()

# Read a header
@app.get("/items/")
def read_items(user_agent: Optional[str] = Header(None)):
    return {"User-Agent": user_agent}

# Custom header
@app.get("/check-auth/")
def check_auth(x_token: str = Header(...)):
    if x_token != "my-secret-token":
        return {"error": "Invalid token"}
    return {"message": "Authenticated"}

# Duplicate headers (multiple values)
from typing import List

@app.get("/headers/")
def read_headers(x_tag: Optional[List[str]] = Header(None)):
    return {"X-Tag values": x_tag}

# Common headers
@app.get("/request-info/")
def request_info(
    host: str = Header(...),
    accept: str = Header(None),
    accept_language: str = Header(None),
    authorization: str = Header(None)
):
    return {
        "host": host,
        "accept": accept,
        "language": accept_language,
```

```
"has_auth": authorization is not None  
}
```

 **Exercise:** Create an endpoint that reads an `X-API-Key` header. Validate it against a hardcoded key and return 401 if invalid.

21. Response Model

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import Optional

app = FastAPI()

class ItemIn(BaseModel):
    name: str
    description: str = None
    price: float
    tax: float = None
    internal_code: str # hidden from response

class ItemOut(BaseModel):
    name: str
    price: float
    tax: float = None

@app.post("/items/", response_model=ItemOut)
def create_item(item: ItemIn):
    return item # internal_code is filtered out

# response_model_exclude_unset
@app.get("/items/{id}", response_model=ItemOut, response_model_exclude_unset=True)
def get_item(id: int):
    return {"name": "Item", "price": 10.0}

# response_model_include / exclude
@app.get("/items/", response_model=list[ItemOut])
def list_items():
    return [{"name": "A", "price": 1.0}]
```

 **Exercise:** Create In/Out models for a User. Input has password, output excludes password. Create a POST endpoint that returns only the safe fields.

[← Back to Tutorials](#)

22. Nested Models

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import List, Optional

app = FastAPI()

# Sub-model
class Image(BaseModel):
    url: str
    name: str

# List of sub-models
class Item(BaseModel):
    name: str
    price: float
    tags: List[str] = []
    images: List[Image] = [] # nested list

# Deep nesting
class Address(BaseModel):
    street: str
    city: str
    zip: str

class User(BaseModel):
    username: str
    address: Address # nested object
    items: List[Item] = [] # list of nested

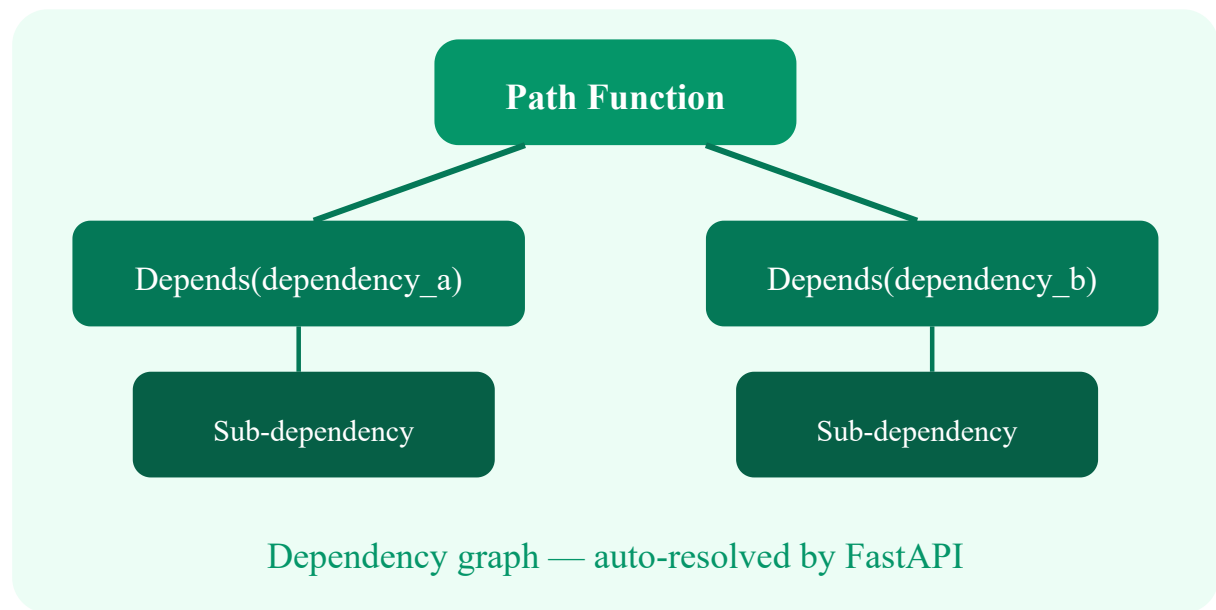
@app.post("/users/")
def create_user(user: User):
    return user

# Arbitrary dicts
from pydantic import Field
```

```
class Product(BaseModel):  
    name: str  
    metadata: dict = {} # arbitrary key-value
```

 **Exercise:** Create models for a Blog with Author (name, bio), Post (title, content, tags[]), and Comment (author, text, date). Create a POST endpoint that accepts a full blog post.

23. Dependencies



```

from fastapi import FastAPI, Depends, Header, HTTPException

app = FastAPI()

# Simple dependency
def common_params(q: str = None, skip: int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}

@app.get("/items/")
def read_items(common_params: dict = Depends(common_params)):
    return common_params

# Dependency with validation
def verify_token(x_token: str = Header(...)):
    if x_token != "secret":
        raise HTTPException(status_code=401)
    return x_token

@app.get("/protected/")
def protected(token: str = Depends(verify_token)):
    return {"message": "Access granted"}

# Class-based dependency
class CommonQueryParams:
    def __init__(self, q: str = None, skip: int = 0, limit: int = 100):
        self.q = q
        self.skip = skip
        self.limit = limit

@app.get("/users/")
def read_users(common_params: CommonQueryParams = Depends()):
    return common_params

# Sub-dependencies (Dependency graph)
def query_extractor(q: str = None):
    return q

def query_or_body_extractor(q: str = Depends(query_extractor)):
    return q or "default"

```

```
@app.get("/search/")
def search(query: str = Depends(query_or_body_extractor)):
    return {"query": query}
```

 **Exercise:** Create a dependency that validates an API key from headers. Use it to protect 3 endpoints. Add a second dependency for pagination (page, size).

24. CORS (Cross-Origin Resource Sharing)

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

# Allow all origins (development only)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Specific origins (production)
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "https://myfrontend.com",
        "https://app.myfrontend.com",
    ],
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE"],
    allow_headers=["Authorization", "Content-Type"],
    expose_headers=["X-Total-Count"],
    max_age=600,
)

# Why CORS?
# Browsers block cross-origin requests by default.
# CORS headers tell the browser: "This server allows your origin."
```

 **Note:** Use `allow_origins=["*"]` only during development. In production, list specific origins to prevent malicious sites from accessing your API.

 **Exercise:** Set up CORS to allow only your localhost:3000 frontend. Test with a simple `fetch()` from a different port — it should be blocked.

[← Back to Tutorials](#)

25. CRUD Operations

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import List, Optional

app = FastAPI()

class Item(BaseModel):
    id: Optional[int] = None
    name: str
    price: float
    is_offer: bool = False

# In-memory database
items_db = {}
counter = 0

# CREATE
@app.post("/items/", status_code=201)
def create_item(item: Item):
    global counter
    counter += 1
    item.id = counter
    items_db[item.id] = item
    return item

# READ (all)
@app.get("/items/")
def list_items(skip: int = 0, limit: int = 10):
    return list(items_db.values())[skip:skip+limit]

# READ (one)
@app.get("/items/{item_id}")
def get_item(item_id: int):
    if item_id not in items_db:
        raise HTTPException(404, "Item not found")
    return items_db[item_id]
```

```
# UPDATE
@app.put("/items/{item_id}")
def update_item(item_id: int, item: Item):
    if item_id not in items_db:
        raise HTTPException(404, "Item not found")
    item.id = item_id
    items_db[item_id] = item
    return item

# DELETE
@app.delete("/items/{item_id}", status_code=204)
def delete_item(item_id: int):
    if item_id not in items_db:
        raise HTTPException(404, "Item not found")
    del items_db[item_id]
```

 **Exercise:** Build a complete CRUD API for a Todo app (title, description, completed, due_date). Test all 5 operations in Swagger UI.

[← Back to Tutorials](#)

26. SQL Databases

```
pip install sqlalchemy databases asyncpg psycopg2-binary alembic
```

SQLAlchemy Setup

```
# database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

SQLALCHEMY_DATABASE_URL = "sqlite:///./test.db"
# Postgres: "postgresql://user:pass@localhost/db"

engine = create_engine(SQLALCHEMY_DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()
```

Model

```
# models.py
from sqlalchemy import Column, Integer, String, Float, Boolean
from database import Base

class Item(Base):
    __tablename__ = "items"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, index=True)
    price = Column(Float)
    is_offer = Column(Boolean, default=False)
```

Dependency

```
# main.py
from fastapi import Depends, FastAPI
from sqlalchemy.orm import Session
from database import SessionLocal, engine
import models

models.Base.metadata.create_all(bind=engine)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/items/")
def read_items(skip: int = 0, limit: int = 10, db: Session = Depends(get_db)):
    return db.query(models.Item).offset(skip).limit(limit).all()
```

 **Exercise:** Set up SQLAlchemy with SQLite. Create a User model (id, username, email). Implement CRUD endpoints using the DB session dependency.

[← Back to Tutorials](#)

27. MongoDB

```
pip install motor # async MongoDB driver
```

Connection Setup

```
# database.py
from motor.motor_asyncio import AsyncIOMotorClient

MONGO_URI = "mongodb://localhost:27017"
client = AsyncIOMotorClient(MONGO_URI)
db = client["fastapi_tutorial"]
items_collection = db["items"]
```

Pydantic Models

```
from pydantic import BaseModel
from typing import Optional
from bson import ObjectId

class Item(BaseModel):
    name: str
    price: float
    is_offer: bool = False

class ItemInDB(Item):
    id: str # MongoDB ObjectId as string
```

Async CRUD

```
from fastapi import FastAPI, HTTPException
from database import items_collection

app = FastAPI()

@app.post("/items/", status_code=201)
async def create_item(item: Item):
    result = await items_collection.insert_one(item.dict())
    created = await items_collection.find_one({"_id": result.inserted_id})
    created["id"] = str(created.pop("_id"))
    return created

@app.get("/items/{item_id}")
async def get_item(item_id: str):
    item = await items_collection.find_one({"_id": ObjectId(item_id)})
    if not item:
        raise HTTPException(404)
    item["id"] = str(item.pop("_id"))
    return item
```

 **Exercise:** Set up Motor with MongoDB. Create a Product model and implement full CRUD. Test with Swagger UI.

[← Back to Tutorials](#)

28. GraphQL

```
pip install strawberry-graphql
```


Strawberry Setup

```
import strawberry
from fastapi import FastAPI
from strawberry.fastapi import GraphQLRouter

# Schema definition
@strawberry.type
class Item:
    id: int
    name: str
    price: float

@strawberry.type
class Query:
    @strawberry.field
    def items(self, skip: int = 0, limit: int = 10) -> List[Item]:
        return [Item(id=1, name="Sample", price=10.0)]

    @strawberry.field
    def item(self, id: int) -> Item:
        return Item(id=id, name="Item", price=20.0)

@strawberry.type
class Mutation:
    @strawberry.mutation
    def create_item(self, name: str, price: float) -> Item:
        return Item(id=2, name=name, price=price)

schema = strawberry.Schema(query=Query, mutation=Mutation)
graphql_app = GraphQLRouter(schema)

app = FastAPI()
app.include_router(graphql_app, prefix="/graphql")
```

Test GraphQL

```
# Query at http://localhost:8000/graphql

{
  items(skip: 0, limit: 10) {
    id
    name
    price
  }
}

# Mutation
mutation {
  createItem(name: "New Item", price: 15.0) {
    id
    name
  }
}
```

 **Exercise:** Create a GraphQL schema with User type (id, username, email). Add queries for users and user(id). Add a mutation to create a user.

29. WebSockets

```
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from typing import List

app = FastAPI()

class ConnectionManager:
    def __init__(self):
        self.active_connections: List[WebSocket] = []

    async def connect(self, websocket: WebSocket):
        await websocket.accept()
        self.active_connections.append(websocket)

    def disconnect(self, websocket: WebSocket):
        self.active_connections.remove(websocket)

    async def broadcast(self, message: str):
        for connection in self.active_connections:
            await connection.send_text(message)

manager = ConnectionManager()

@app.websocket("/ws/{client_id}")
async def websocket_endpoint(websocket: WebSocket, client_id: str):
    await manager.connect(websocket)
    try:
        while True:
            data = await websocket.receive_text()
            await manager.broadcast(f"Client {client_id}: {data}")
    except WebSocketDisconnect:
        manager.disconnect(websocket)
        await manager.broadcast(f"Client {client_id} left")
```

Client Example (HTML/JS)

```
<script>
  const ws = new WebSocket("ws://localhost:8000/ws/user1");
  ws.onmessage = (event) => console.log(event.data);
  ws.send("Hello everyone!");
</script>
```

 **Exercise:** Build a simple chat server using WebSockets. Create an HTML client that connects, sends messages, and displays incoming messages in real time.

30. Event Handlers

Startup & Shutdown (Legacy)

```
from fastapi import FastAPI

app = FastAPI()

@app.on_event("startup")
async def startup_event():
    print("Starting up...")
    # Initialize DB connection
    # Load ML model
    # Create connection pool

@app.on_event("shutdown")
async def shutdown_event():
    print("Shutting down...")
    # Close DB connections
    # Clean up resources
```

Lifespan (Modern)

```
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup
    print("Server starting...")
    app.state.db = await create_db_pool()
    app.state.ml_model = load_model()
    yield
    # Shutdown
    print("Server shutting down...")
    await app.state.db.close()

app = FastAPI(lifespan=lifespan)

@app.get("/predict/")
async def predict(text: str):
    result = app.state.ml_model.predict(text)
    return {"prediction": result}
```

Use Cases

- Database connection pool setup/teardown
- Loading ML/AI models
- Creating HTTP client sessions
- Initializing cache
- Registering background tasks

 **Exercise:** Use the lifespan pattern to initialize a database connection pool on startup and close it on shutdown. Use `app.state` to store the pool.

[← Back to Tutorials](#)

31. Sub-Applications

```
from fastapi import FastAPI, APIRouter

app = FastAPI()

# Create sub-app
subapp = FastAPI(title="Sub API")

@subapp.get("/sub/items/")
def sub_items():
    return {"message": "This is the sub-app"}

# Mount sub-app at prefix
app.mount("/sub", subapp)

# Using APIRouter (better for organizing)
router = APIRouter(prefix="/api/v1", tags=["v1"])

@router.get("/items/")
def v1_items():
    return {"version": "v1", "items": []}

@router.get("/users/")
def v1_users():
    return {"version": "v1", "users": []}

app.include_router(router)

# Versioning example
v2_router = APIRouter(prefix="/api/v2", tags=["v2"])

@v2_router.get("/items/")
def v2_items():
    return {"version": "v2", "items": [], "new_field": "extended"}

app.include_router(v2_router)
```


Mounting Starlette Apps

```
from starlette.applications import Starlette

starlette_app = Starlette()
# ... configure starlette_app
app.mount("/starlette", starlette_app)
```

 **Exercise:** Create a main app with two versioned routers (v1 and v2). Each has /items and /users endpoints but v2 returns additional fields.

[← Back to Tutorials](#)

32. Middleware

```
from fastapi import FastAPI, Request
from starlette.middleware.base import BaseHTTPMiddleware
import time

app = FastAPI()

# Custom middleware class
class ProcessTimeMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        start = time.time()
        response = await call_next(request)
        process_time = time.time() - start
        response.headers["X-Process-Time"] = str(process_time)
        return response

app.add_middleware(ProcessTimeMiddleware)

# Simple middleware using @app.middleware
@app.middleware("http")
async def add_custom_header(request: Request, call_next):
    response = await call_next(request)
    response.headers["X-Powered-By"] = "FastAPI"
    return response

# Authentication middleware
@app.middleware("http")
async def authenticate(request: Request, call_next):
    if request.url.path.startswith("/admin"):
        token = request.headers.get("Authorization")
        if not token:
            return JSONResponse(
                {"error": "Not authenticated"},
                status_code=401
            )
    response = await call_next(request)
    return response
```

 **Exercise:** Create middleware that logs each request's method, path, and duration.
Add a middleware that blocks requests from specific IPs.

[← Back to Tutorials](#)

33. Flask Mount

```
pip install flask
```

```
from fastapi import FastAPI
from flask import Flask, jsonify

app = FastAPI()

# Create a Flask app
flask_app = Flask(__name__)

@flask_app.route("/flask/hello")
def flask_hello():
    return jsonify({"message": "Hello from Flask!"})

@flask_app.route("/flask/items/")
def flask_item(item_id):
    return jsonify({"item_id": item_id, "source": "Flask"})

# Mount Flask app under a prefix
app.mount("/legacy", flask_app)

# FastAPI endpoint alongside Flask
@app.get("/fastapi/hello")
def fastapi_hello():
    return {"message": "Hello from FastAPI!"}

# The Flask app is served at /legacy/flask/...
# The FastAPI app serves /fastapi/...
```

When to Mount Flask

- Gradual migration from Flask to FastAPI

- Reusing legacy Flask code without rewriting
- Running both frameworks side by side during transition

 **Note:** Flask runs via WSGI while FastAPI is ASGI. FastAPI automatically wraps Flask for ASGI compatibility when mounting.

 **Exercise:** Create a Flask app with 3 endpoints (users, posts, comments). Mount it on a FastAPI app. Add one new FastAPI endpoint alongside it.

[← Back to Tutorials](#)

34. Deployment

Uvicorn + Gunicorn (Production)

```
pip install gunicorn uvicorn

# Run with gunicorn workers
gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker

# With systemd service
[Unit]
Description=FastAPI Application
After=network.target

[Service]
User=www-data
WorkingDirectory=/opt/app
ExecStart=/usr/bin/gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker
Restart=always

[Install]
WantedBy=multi-user.target
```

Docker

```
# Dockerfile
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

EXPOSE 8000
CMD ["gunicorn", "main:app", "-w", "4", "-k", "uvicorn.workers.UvicornWorker"]

# docker-compose.yml
services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://user:pass@db/db
    depends_on:
      - db
  db:
    image: postgres:15
```

Cloud Hosting Options

Provider	Notes
Railway	Easy deploy, auto HTTPS, domains
Render	Free tier, auto-deploy from Git
Fly.io	Edge deployment, global regions
AWS (ECS/EKS)	Docker containers, auto-scaling
Azure Container Apps	Serverless containers, built-in scaling

 **Exercise:** Create a Dockerfile for your FastAPI app. Run it locally with docker-compose including a PostgreSQL service. Test that the API works inside the container.

35. Quick Guide

Task	Command / Code
Install	<code>pip install fastapi uvicorn</code>
Run dev server	<code>uvicorn main:app --reload</code>
Run production	<code>gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker</code>
Create app	<code>app = FastAPI()</code>
GET endpoint	<code>@app.get("/path")</code>
POST endpoint	<code>@app.post("/path")</code>
Path param	<code>def func(item_id: int)</code>
Query param	<code>def func(q: str = None)</code>
Request body	<code>def func(item: Item)</code>
Form data	<code>def func(name: str = Form(...))</code>
File upload	<code>def func(file: UploadFile = File(...))</code>
Dependency	<code>def func(db: Session = Depends(get_db))</code>
Response model	<code>@app.get("/path", response_model=ItemOut)</code>
Status code	<code>@app.post("/path", status_code=201)</code>
Docs URL	<code>/docs (Swagger), /redoc (ReDoc)</code>
Mount static	<code>app.mount("/static", StaticFiles(...))</code>
CORS	<code>app.add_middleware(CORSMiddleware, ...)</code>

 **Exercise:** Use this quick guide as a cheatsheet for your next FastAPI project. Try building a complete API without looking at the full chapters.

36. Useful Resources

Official Docs & Learning

- **FastAPI Docs:** fastapi.tiangolo.com — Official tutorial, advanced guides, deployment
- **Starlette Docs:** starlette.io — ASGI framework under FastAPI
- **Pydantic Docs:** docs.pydantic.dev — Data validation library

Community

- **FastAPI GitHub:** github.com/tiangolo/fastapi — Issues, discussions, examples
- **Discord:** FastAPI community chat
- **Stack Overflow:** Tagged questions for troubleshooting

Tools & Ecosystem

- **SQLModel:** SQL databases with Pydantic + SQLAlchemy
- **FastAPI Users:** Authentication and authorization
- **FastAPI CRUD:** Automatic CRUD generation
- **Alembic:** Database migrations
- **Pytest + httpx:** Async testing

37. Discussion

Q&A

Q: FastAPI vs Flask — which should I learn?

Both. Flask for simpler apps and microservices. FastAPI for APIs needing async, auto-docs, and validation. Many teams use both.

Q: Is FastAPI production-ready?

Yes. Used by Microsoft, Uber, Netflix, and many startups. It's as production-ready as Flask or Django.

Q: Should I use async or sync endpoints?

Use async for I/O-bound tasks (DB queries, HTTP calls, file I/O). Use sync for CPU-bound tasks. FastAPI handles both.

Q: How do I test FastAPI apps?

Use `pytest` with `httpx.AsyncClient` or FastAPI's `TestClient` (based on Starlette).

Advanced Topics

- Background tasks with `BackgroundTasks`
- Rate limiting with middleware
- OpenAPI extensions and custom generators
- gRPC integration via `grpclib`
- Async SQLAlchemy 2.0

38. Appendix

Best Practices

- Use `APIRouter` to organize endpoints by domain
- Separate models (schemas), services, and routes
- Use dependency injection for DB sessions, auth, and config
- Always validate input with Pydantic models
- Use `response_model` to control output
- Set up CORS explicitly for production
- Use environment variables for configuration
- Add logging and request ID middleware

Interview Questions

1. What is FastAPI and how does it differ from Flask?
2. Explain the role of Pydantic in FastAPI.
3. How does FastAPI handle async requests?
4. What is dependency injection in FastAPI?
5. How do you validate query parameters?
6. Explain the lifespan protocol.
7. How would you mount a Flask app inside FastAPI?
8. What is CORS and how do you configure it?
9. How do you handle database sessions in FastAPI?
10. What are WebSockets and how do you implement them?

Glossary

Term	Definition
ASGI	Asynchronous Server Gateway Interface — standard for async Python web apps
WSGI	Web Server Gateway Interface — synchronous standard (Flask, Django)
Pydantic	Data validation library using Python type annotations
OpenAPI	Standard API specification format (formerly Swagger)
Starlette	ASGI framework that FastAPI is built on
Uvicorn	ASGI server for running FastAPI

39. Bonus: Final Wrap-Up

What You've Learned

- FastAPI fundamentals: routing, parameters, validation, responses
- Pydantic models for data validation and serialization
- Templates, static files, forms, and file uploads
- Cookies, headers, response models, nested models
- Dependency injection, CORS, middleware, sub-apps
- SQL databases (SQLAlchemy), MongoDB (Motor), GraphQL (Strawberry)
- WebSockets for real-time communication
- Event handlers and the lifespan protocol
- Deployment with Docker, Gunicorn, and cloud providers

Who's Who in FastAPI Ecosystem

Person / Project	Role
Sebastián Ramírez (tiangolo)	Creator of FastAPI — also created Typer, SQLAlchemy
Tom Christie	Creator of Starlette — ASGI framework under FastAPI
Samuel Colvin	Creator of Pydantic — data validation
Encode OSS	Organization behind Starlette, httpx, databases
FastAPI Community	100K+ GitHub stars, active Discord, 500+ contributors

Final Project

Build a complete e-commerce API with FastAPI featuring product CRUD, user auth, cart management, order history, and real-time notifications via WebSockets. Containerize it with Docker and deploy to a cloud provider.

 **Exercise:** Complete the final project above. Deploy it to Railway or Render and share the URL. Congratulations on finishing the course!